

Práctica 7 – Construir el módulo Miembros

Tópico de Aplicaciones Web · Unidad II

Alumno: Isaac Andre Arellano 229842

Este documento funciona como README de la Práctica 7. Incluye la descripción breve del trabajo realizado, las respuestas solicitadas y espacios para insertar las cinco capturas de evidencia de las peticiones del módulo Miembros.

Objetivo

Construir el módulo Miembros aplicando el mismo patrón utilizado previamente en Inscripciones: dominio, interfaz de repositorio, implementación en memoria, token de inyección, Service, Controller y Module. El módulo debe integrarse sin afectar el funcionamiento de Inscripciones.

Estructura implementada

El módulo Miembros contiene la entidad Miembro, MiembroRepository, los DTO para crear y actualizar, MiembroMemoriaRepository, el token MIEMBRO_REPOSITORY, MiembrosService, MiembrosController y MiembrosModule.

Preguntas de la práctica

1. ¿Por qué esta interfaz no menciona Express, NestJS ni memoria?

Porque MiembroRepository pertenece al dominio y únicamente define el contrato de las operaciones que se pueden realizar con los miembros. No depende de la forma en que llegan las peticiones ni de dónde se almacenan los datos. Esto permite cambiar la implementación del repositorio sin modificar el contrato del dominio.

2. ¿Qué palabra de MiembroMemoriaRepository promete cumplir la interfaz del paso anterior?

La palabra es implements. Al declarar MiembroMemoriaRepository implements MiembroRepository, la clase se compromete a proporcionar los métodos definidos por la interfaz.

3. ¿Por qué miembros.service.ts no sabe qué es una petición HTTP?

Porque el Service se encarga de la lógica de aplicación y de delegar las operaciones al repositorio. Los detalles de HTTP, como rutas, parámetros y cuerpos de petición, pertenecen al Controller. De esta manera se mantienen separadas las responsabilidades.

4. ¿Por qué el Service se inyecta sin token en el Controller, y el repositorio sí necesita uno?

MiembrosService es una clase concreta que existe en tiempo de ejecución y NestJS puede usar esa clase directamente como token de inyección. MiembroRepository es una interfaz de TypeScript y desaparece al compilar, por lo que se utiliza MIEMBRO_REPOSITORY como token para indicar qué implementación concreta debe inyectarse.

5. ¿Qué prueba, en los hechos, que agregar Miembros no rompió nada de Inscripciones?

La comprobación práctica consiste en levantar la aplicación, verificar que las rutas de Miembros e Inscripciones estén registradas y volver a ejecutar una petición de Inscripciones. Si Inscripciones sigue respondiendo correctamente después de integrar Miembros, se demuestra que el nuevo módulo no rompió su funcionamiento.

Evidencias

Insertar debajo de cada apartado la captura completa de la respuesta obtenida con REST Client.

Evidencia 1 – Listar miembros

Petición: GET /miembros

Resultado esperado: lista de los tres miembros iniciales.

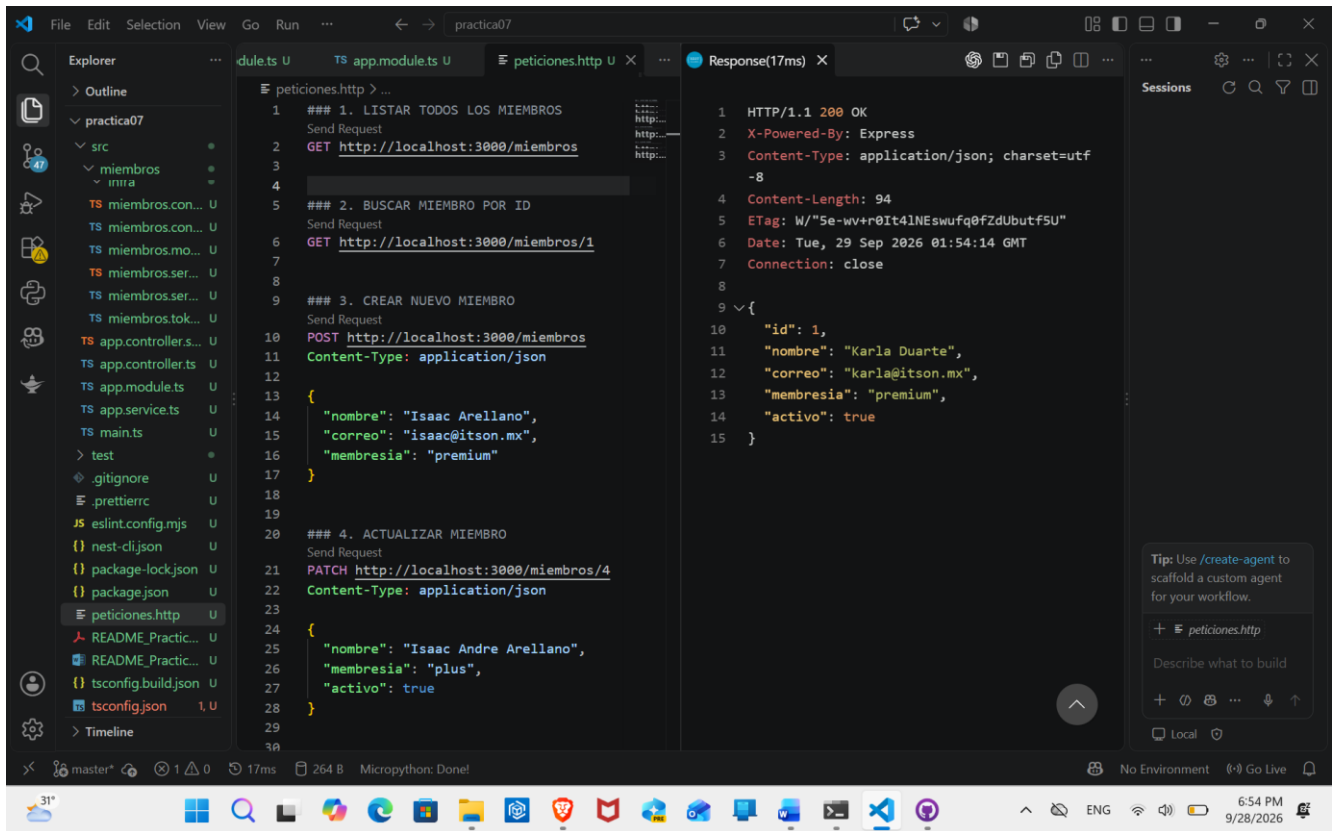
```
1 ## 1. LISTAR TODOS LOS MIEMBROS
2 GET http://localhost:3000/miembros
3
4
5 ## 2. BUSCAR MIEMBRO POR ID
6 GET http://localhost:3000/miembros/1
7
8
9 ## 3. CREAR NUEVO MIEMBRO
10 POST http://localhost:3000/miembros
11 Content-Type: application/json
12 {
13   "nombre": "Isaac Arellano",
14   "correo": "isaac@itson.mx",
15   "membresia": "premium"
16 }
17
18
19 ## 4. ACTUALIZAR MIEMBRO
20 PATCH http://localhost:3000/miembros/4
21 Content-Type: application/json
22 {
23   "nombre": "Isaac Andre Arellano",
24   "membresia": "plus",
25   "activo": true
26 }
27
28
29
```

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 280
5 ETag: W/"118-TX8su0DnBBRagxScJCcZ6o91u10"
6 Date: Tue, 29 Sep 2026 01:53:25 GMT
7 Connection: close
8
9 [
10   {
11     "id": 1,
12     "nombre": "Karla Duarte",
13     "correo": "karla@itson.mx",
14     "membresia": "premium",
15     "activo": true
16   },
17   {
18     "id": 2,
19     "nombre": "Omar Valdez",
20     "correo": "omar@itson.mx",
21     "membresia": "plus",
22     "activo": true
23   },
24   {
25     "id": 3,
26     "nombre": "Sofia Ibarra",
27     "correo": "sofia@itson.mx",
28     "membresia": "basica",
29     "activo": true
30   }
31 ]
```

Evidencia 2 – Buscar miembro por ID

Petición: GET /miembros/1

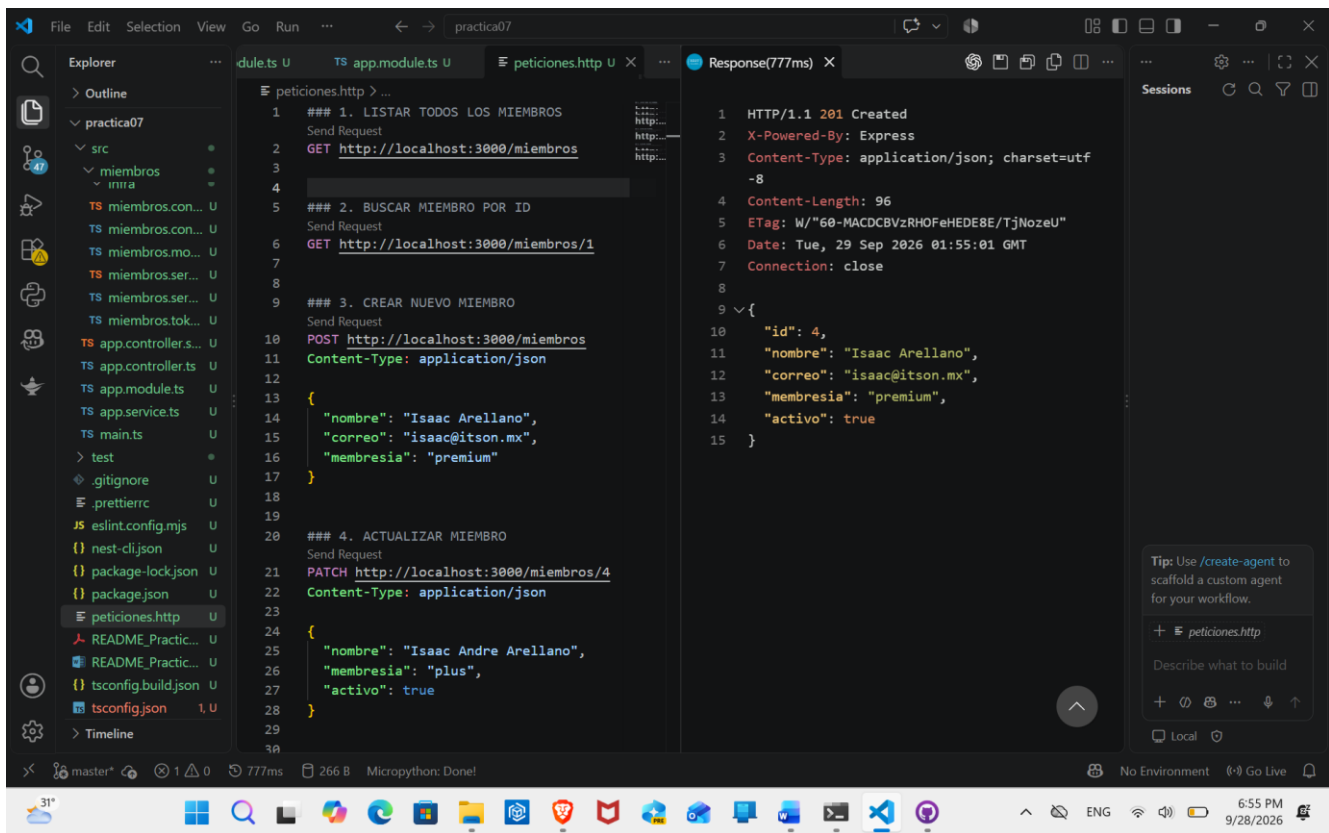
Resultado esperado: información del miembro con id 1.



Evidencia 3 – Crear miembro

Petición: POST /miembros

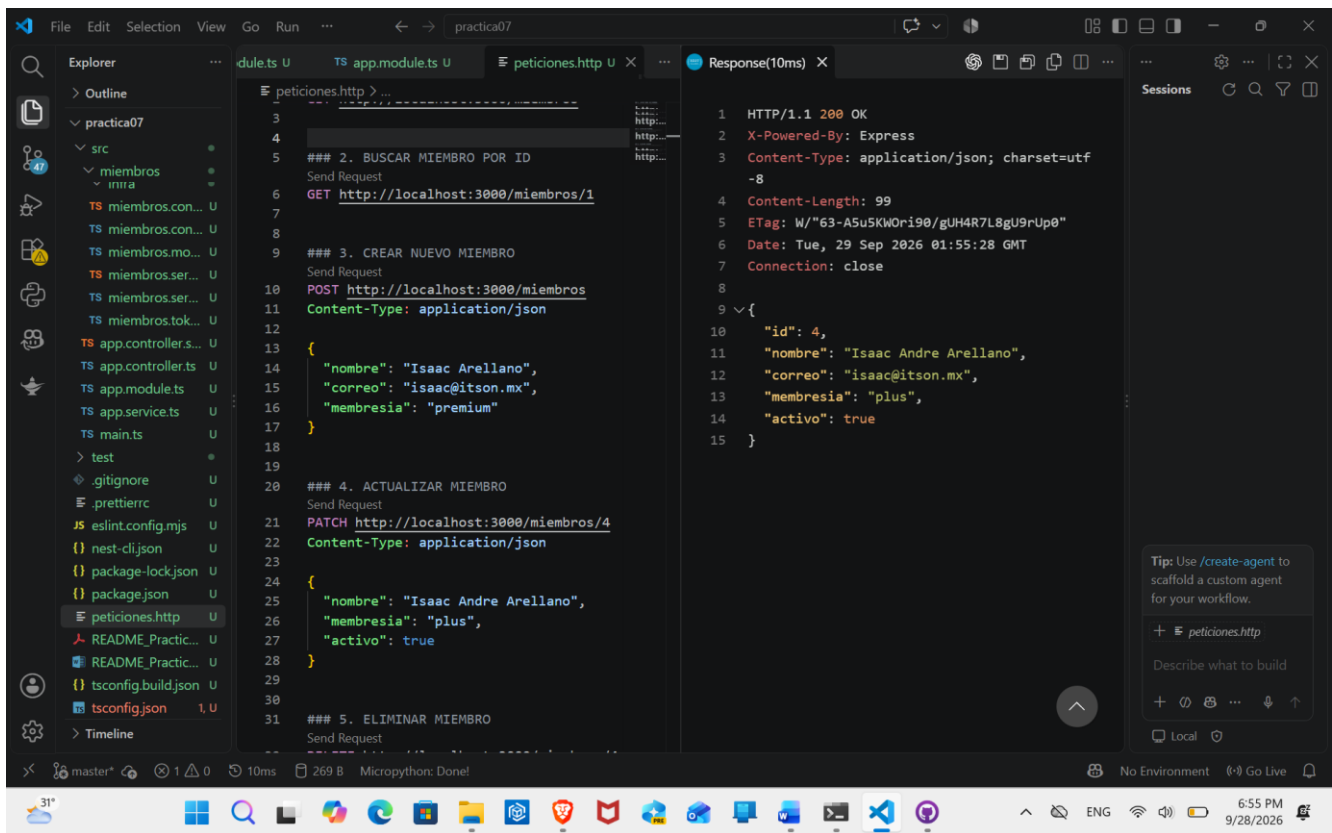
Resultado esperado: creación de un nuevo miembro, que inicia con id 4.



Evidencia 4 – Actualizar miembro

Petición: PATCH /miembros/4

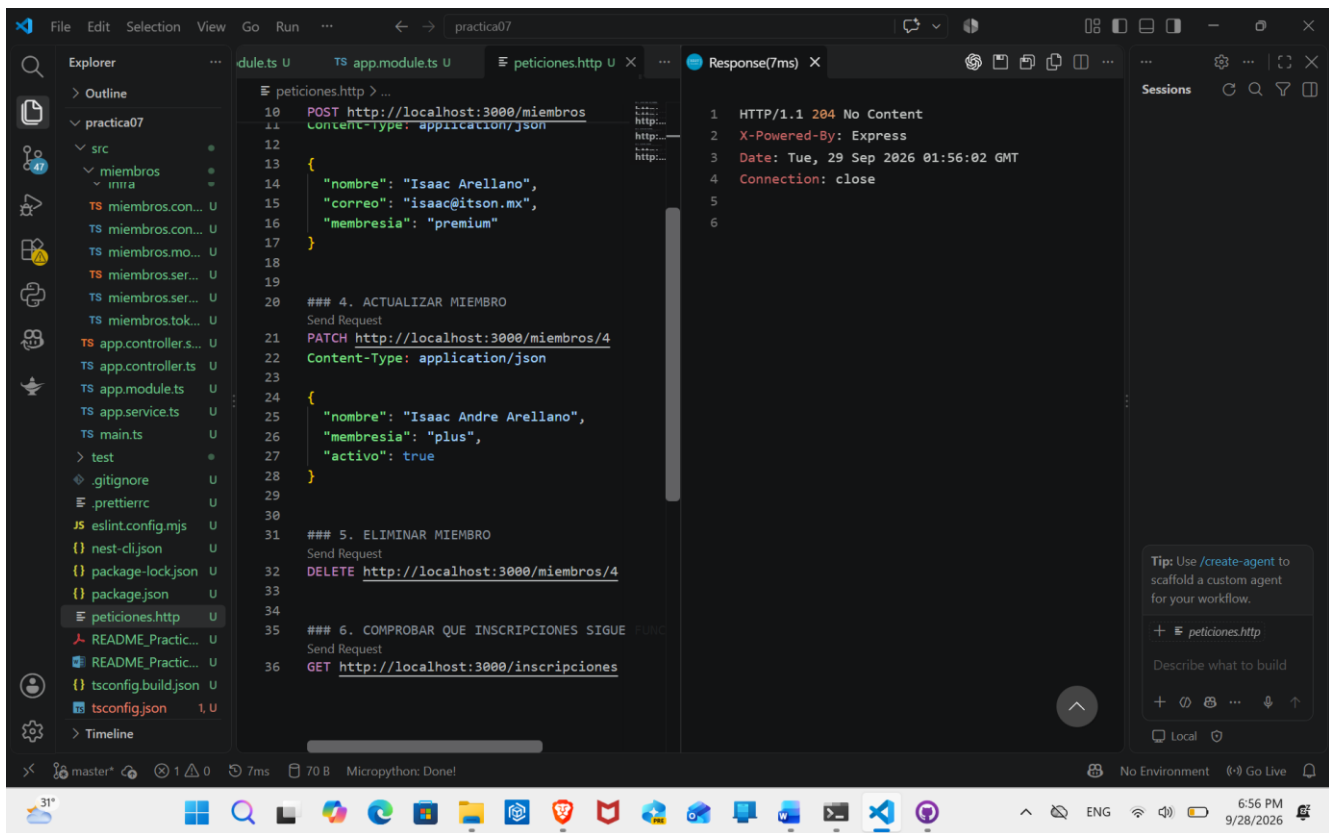
Resultado esperado: datos actualizados del miembro con id 4.



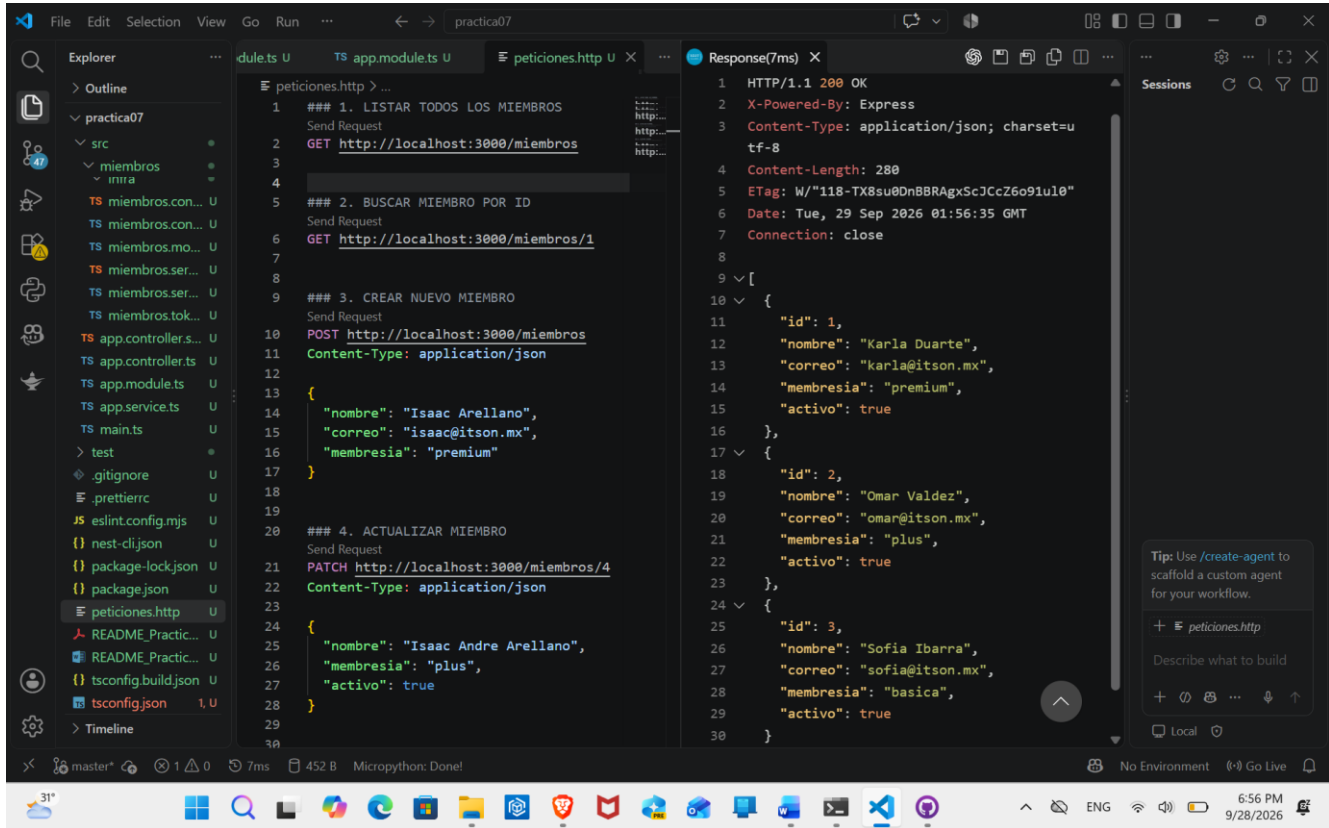
Evidencia 5 – Eliminar miembro

Petición: DELETE /miembros/4

Resultado esperado: petición de eliminación procesada correctamente.

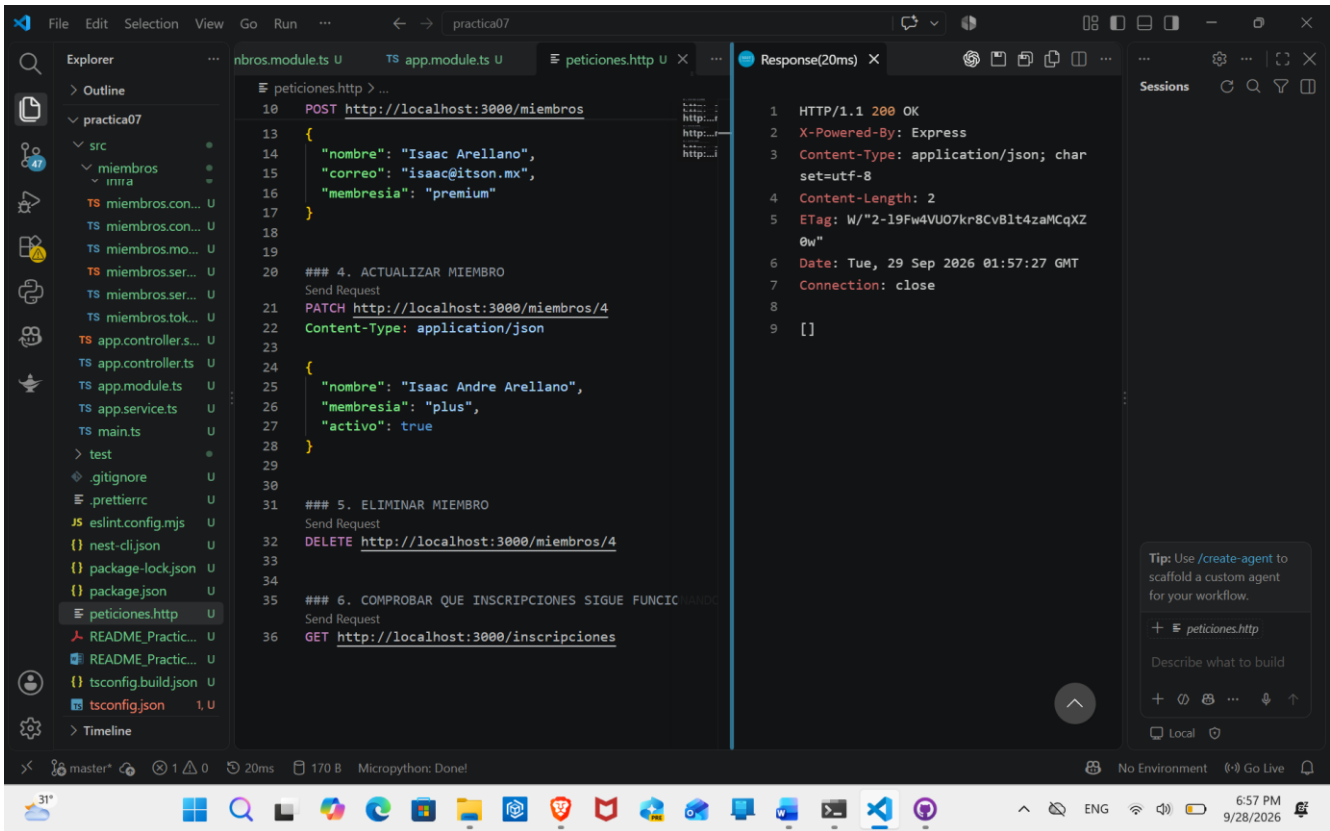


Eliminado correctamente(ya no aparece en la lista)



Comprobación de Inscripciones

Después de probar las cinco operaciones de Miembros, ejecutar nuevamente GET /inscripciones y confirmar que el módulo continúa respondiendo correctamente. Esta comprobación respalda la respuesta de la pregunta 5.



Repositorio

Repositorio: Frontend-Web-Practicas

Carpeta de entrega: practica07

Enlace del repositorio: <https://github.com/ArellanoAndre/Frontend-Web-Practicass.git>